UCF Physics: AST 5765/4762: (Advanced) Astronomical Data Analysis Survey
Homework 4: Python practice part 2
Due Thursday Sep 5, 6pm.

Goals: Become sufficiently familiar with Python to:

1. Display images and 2D plots
2. Write and document functions and procedures with good coding style.

Handy resources:

1. np.lookfor("keyword"): look up functions by word or phrase in docstring
2. remember that help() will give you the docstring of any function
3. Chapter1 -4 of https://ssb.stsci.edu/perry/pydatatut.pdf
4. ThinkPython 2e https://greenteapress.com/wp/think-python-2e/

Remember that you can find information about our coding style under `handouts/homework_format_rules.pdf`.

Do not use loops for any problem on this assignment. Remember to open your `0-ast4762-`*`yourname`*`.log` and start logging what you are doing. Also, remember that '`yourusername`' should be replaced by the username you have chosen.

1. (**5 points**) Use your terminal to make a new folder `hw4_yourname/` under your `ast4762/homework/` folder. Explain in your log how you did it. For this homework your main homework file is a Python file named `hw4_<username>.py` (if you prefer to work on a notebook you will start with a `hw4_<username>.ipynb` but remember to save it as a .py before submission; see end of homework too). Save it in your homework (`hw4/`) folder. Remember to commit your files and push to GitHub as often as needed (also, great backup!).

Your name, assignment number, and the date should appear as comments at the top of the file. At the start of every problem print the problem number using the appropriate Python command. Any remarks or written answers you may make should be printed out using the appropriate Python command. If you need to comment something in the code (for coding clarity) do so with a normal comment (i.e., `# this is a comment`). Remember that using comments is good coding practice, and it will be taken into account during the grading process.

2. (**36 points total**) You will write a function called `square` that returns the square of its input (which may be a scalar or an array of any dimension or numerical type).

We will start with creating your support functions file.

(a) (**2 points**) Copy the template we provide in `module_template_with_docstring.py` from our `homework/HW4/` folder to

create the content of your support functions file. Save your support file following the naming conventions of the class (see 2.1).

(b) (**3 points**) Study the structure of the template carefully. This is how you want your support file to be structured ideally. What are the parts that need to be included? Describe them and explain why you think they are there in your log.

(c) (**5 points**) Make the template suitable for this homework assignment. Replace and remove imports as needed. You will be cleaning up this file more in the following parts. Note that this file will contain multiple functions. Failure to clean the file up properly will result in loss of points.

Now we will start writing our first function of this class.

(d) (**2 points**) Start the first function of the file named `square`.
`square` takes as input a scalar or array of any dimension or numerical type and returns its square.
Commit to GitHub. Explain how you did all this in your log.

(e) (**8 points**) Write the code for your function `square`. Document the function following the template as well as you can. Be sure to include *at least one* example in your docstring!

(f) (**8 points**) Test and iterate until your code works. Document in your log bugs you find, and the test cases you used.
Commit to Git/GitHub every time you do a major update to your code. Remember to *test at least one example of all possible input data types*!

(g) (**2 points**) Update the example and docstring as needed. Commit one final time to GitHub. Make a screenshot of the Git log history showing the updates you did to your file. Name the screenshot appropriately and save it in your homework folder.

Use your function!

(h) (**3 points**) In your main homework file (`hw4_<username>.py` or `hw4_<username>.ipynb`) create an array `test_square_1` with integers from 0 to 9. Import and call function `square` to square array `test_square_1` and print the result.

(i) (**3 points**) In a single line, create a 5x5 array `test_square_2` that contains floats from 0 to 25. Call function `square` to square array `test_square_2` and print the result.

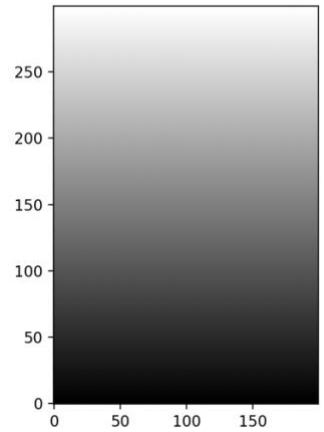
3. (**20 points**) In your support function file write another function called `squareplot`. `Squareplot`'s purpose it to plot the squares of numbers. `squareplot` should have three positional arguments: the low end of the range it will plot, the high end of the range (inclusive!) it will plot, and the number of points to plot over this range. It should also have one optional argument, `saveplot=False`. The function should then:
- create an array `x` of evenly spaced points from the low up-to-and-including the high end of the range,
- call function `square` exactly once to get array `y` that is the square of array `x`,
- plot `y` vs `x`.
- If `saveplot` is not False, the function should save the plot image as a PDF in the file name given by the user through the `saveplot` argument (note that the filename should not be hardcoded in the function!)

The horizontal and vertical axes of the plot should be labeled "Input" and "Output", respectively, and the title of the plot should be "Square Function".

Make sure to document the function (in your function's docstring) and include an example. Remember to commit your function into GitHub as soon as you have a file, and several times as you write it and debug it. Also, remember to document bugs and updates you do in your log.

In the main homework file, call `squareplot` to plot (in the same plot) the squares of the numbers 1, 2.5, 4, 5.5, and 7, and save to an appropriate named file. Remember, that you might need to restart your kernel for the updates in your support functions file to be read by your main homework file.

4. (**10 points**) Make a 300×200 Float64 array where each element contains its own `y` coordinate, starting with y = 0 for row 0 and ending with y = 299 for row 299. Display the array using the matplotlib function for showing the image with a cmap of 'gray'. Examine several randomly placed array elements with Python to be sure the values match the y coordinate in the print statements. Be careful that the final array has the correct data type. Make sure that the lower left corner of your plot is 0,0! Your image should look like this:



Take a screen shot showing both your Python code and plot windows.

5. (**10 points**) Download and read FITS file m42_40min_ir.zip from `homework/HW4/` folder. Don't worry, the fits function knows how to read the zip file and you don't need to unzip it in any way (see also: https://docs.astropy.org/en/stable/io/fits/). This file comes from the NOIRLab FITS liberator project. Plot it using the appropriate matplotlib function using a gray scale colormap. Make sure that the lower left corner of your plot is 0,0! What object is it?

- Annotate the image by placing the object's name and your name as a title. Make sure the title fits within the image boundaries!
- Add appropriate axis labels.
- Use the appropriate python command to save the final plot in a PNG format. Remember to use an appropriate name following the class's conventions.

6. (**5 points**) Prepare and submit your homework. If you worked on a Jupyter notebook, make sure to save it as a.py file. Don't forget to commit and push to GitHub. Explain what you did to do that in your log. Make a screenshot that shows your logging history, showing that you committed the file and add it to your `hw4_<yourname>` folder (remember to use an appropriate name for the screenshot!). Then:

- use a terminal to navigate to `ast4762/`. Explain how you did this in your log
- use the appropriate command to copy the `hw4_yourname` folder from your `homework` into your `handin` folder. Explain how you did this in your log
- go into your `handin/hw4_yourname` and run your .py from the terminal. Does it run without issues?
- write what command (no drag and drop!) you will use to copy your log into your `handin/hw4_yourname`, make a tar.gz of the `handin/hw4_yourname` folder and how you will submit that to Webcourses.
- use the appropriate commands to copy (from the terminal) your log into your `handin/hw4_yourname` folder, and make tar.gz of the folder.
- turn the file in on WebCourses.